

Course Project: Cryptographic Algorithms & Modular Arithmetic

Shamir's Secret Sharing

By Diego Martinez

1 Shamir's Secret Sharing and its purpose

Shamir's Secret Sharing (SSS) algorithm is an encryption algorithm that takes a secret, and split it into different pieces or, in other words, shares with the number of shares is denoted by N . The secret can only be reconstructed if there are enough shares, or meet the threshold of shares, which is denoted by a number K .

Basically, a secret can be split among different people, and those people can only reconstruct the secret if enough people agree to recreate it.

How it can be implemented, polynomials would be used to create the shares and also recreate the secret. Let's say a polynomial $y = ax + b$. The b in this case is the constant, and for SSS it would be the secret that is to be encrypted. Shares would be made by plugging in random numbers of x and finding the y , and that pair (x, y) would be a share. The coefficient(s) can be any random number when creating shares. For decrypting a secret, the polynomial that goes through K points must be found, and again the constant of that polynomial would be the secret. Using modulus can also be used in SSS and will be covered later.

2 Mathematical background

In order to implement SSS, there are some concepts to go over first. The first which is the **Lagrange Polynomial Interpolation**, which is used to find the polynomial of the lowest degree that goes through all of the shares. A Lagrange polynomial of some share's x coordinate x_i , is a polynomial that its y value at that x_i is 1, and is 0 for every other share's, at least the shares inputted, x coordinate.

For example, let's say there are some x coordinates $x = 1, 2, 3, 4$, the Lagrange equation of the x coordinate 2, would be some polynomial such that $f(2) = 1$, and $f(1) = f(3) = f(4) = 0$

After getting the polynomial for that value of x_i , the polynomial would be multiplied by the x_i 's respective y value, and now there would be an equation that still is 0 at all other share's x value, but now goes through the x_i 's share. The same thing will be done for all other shares, where the the Lagrange polynomial for those shares are found, and multiply by their respective y values. After getting all of these Lagrange polynomials and add them together, there would now be a polynomial that goes through all of the shares.

Here are the equations for Lagrange Polynomial Interpolation.

$$l_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}$$

$$f(x) = \sum_{i=0}^{K-1} y_i l_i(x)$$

Looking at the first line, there is a product operator ($\prod$). This means that all of the fractions where the divisor is the value of x_i who's Lagrange equation is being found minus another share's x value, and the dividend is x minus the same share's x value as is being subtracted in the divisor, and all of those fractions are multiplied together.

For example, say that, again, there are some x coordinates $x = 1, 2, 3, 4$, the Lagrange equation of $x = 2$ would be:

$$l_2(x) = \frac{x-1}{2-1} \times \frac{x-3}{2-3} \times \frac{x-4}{2-4}$$

Now looking at the second equation, this is the implementation of multiplying each Lagrange polynomial by their respective y , then adding all Lagrange polynomials to finally get the polynomial that goes through all the shares.

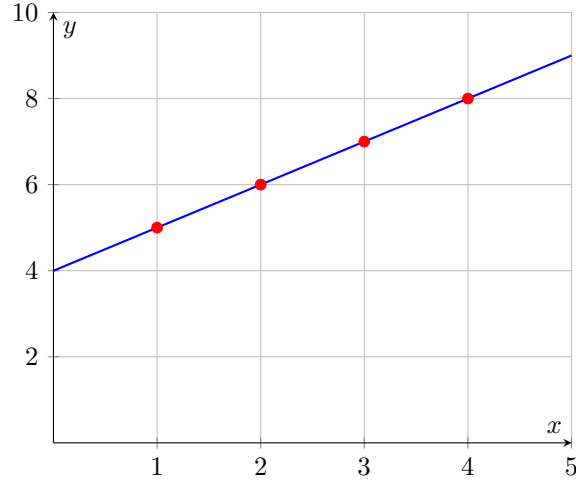
Walking through the full example of $x = 1, 2, 3, 4$, the following Lagrange polynomials are found:

$$\begin{aligned} l_1(x) &= \frac{x-2}{1-2} \times \frac{x-3}{1-3} \times \frac{x-4}{1-4} \\ l_2(x) &= \frac{x-1}{2-1} \times \frac{x-3}{2-3} \times \frac{x-4}{2-4} \\ l_3(x) &= \frac{x-1}{3-1} \times \frac{x-2}{3-2} \times \frac{x-4}{3-4} \\ l_4(x) &= \frac{x-1}{4-1} \times \frac{x-2}{4-2} \times \frac{x-3}{4-3} \end{aligned}$$

Now, multiplying each Lagrange polynomial by their respective y , let it be $y = 5, 6, 7, 8$, then simplifying the equation, the following is found:

$$f(x) = x + 4$$

Plotting the line $f(x) = x + 4$, it does indeed go through the points $(1, 5), (2, 6), (3, 7), (4, 8)$.

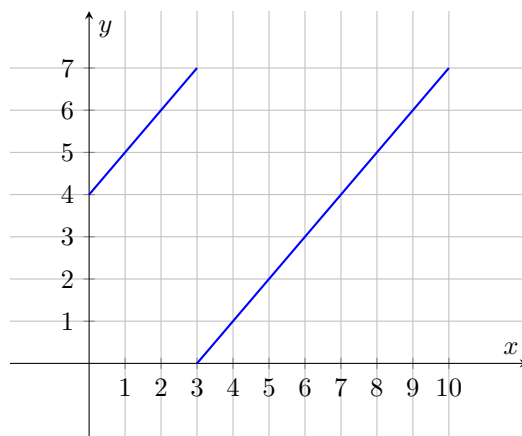


In the case of SSS, we would not use the variable x at all like in the equation $f(x) = x + 4$. Instead, the values of each Lagrange polynomial at $x = 0$ would be looked for, as the constant of the polynomial would be the secret. So the new equations for the use of SSS would be:

$$\begin{aligned} l_i(0) &= \prod_{j=0, j \neq i}^n \frac{-x_j}{x_i - x_j} \\ f(0) &= \sum_{i=0}^{K-1} y_i l_i(0) \end{aligned}$$

The bottom equation would stay the same.

With just the Lagrange Polynomial Interpolation, a basic version of SSS can be achieved, but another concept that could be added to improve the security of the algorithm. This concept is **Modular Arithmetic**. Modular Arithmetic is where the remainder of the division. For example, the remainder of 8 divided by 6 is 2. Equivalently, $8 \bmod 6$ would be 2. When applying this to an equation, for example $(x + 4) \bmod 7$ we get:



Looking at this graph, we can see that the equation $y = x + 4$ loops around under the line $x = 7$. How does this apply to the SSS algorithm? Now that we have a polynomial with the secret of 4 under the modulus of 7 in this case, we make some shares. Let $x = 1, 2, 3, 4$. Plug them into the polynomial and we get $y = 5, 6, 0, 1$. Now the points look scrambled and would be much harder to brute force looking for a polynomial that contains the secret using Lagrange if there are some shares known but not enough to recreate the secret. The divisor number thing will be referred to as modulus in this report, like the number 7 in mod 7 will be referred to as modulus.

One more thing to note about doing mod is that we cannot do regular division under mod. For example, the fraction in $(\frac{3}{7}) \bmod 5$ cannot be done regularly. The way around this is that we have to find the **mod inverse** of the divisor. The mod inverse of a number a is a number b under mod c such that $b \cdot a \bmod c = 1$. For example, the mod inverse of 7 under mod 5 is 3 since $3 \cdot 7 \bmod 5 = 1$. Now that we have the mod inverse, we can multiply the dividend by the mod inverse and that would be how we do division under mod.

3 Pseudocode

Now how can we implement all of this? Let us first look at **encryption**. To encrypt a secret, we need the number that we want to be a secret S , the number of shares N , and the threshold K , making sure that $N \geq K$ since we need enough shares to recreate the secret. The degree of polynomial needed to encrypt the secret would be $K - 1$, so for threshold of 3, we would need a second degree polynomial. for example $ax^2 + bx + c = y$. The coefficients can be any random integer, but let us make sure that all of them are not 0. The secret would again be the constant c . Now to implement the mod stuff, we must choose a prime number to be the modulus to avoid any weirdness when doing the mod calculation. The modulus must also be greater than S, K, N . For this report, we will choose the next biggest prime number of the greatest number between the S, K, N . **Now lets get the steps for encryption:**

1. Input the S, K, N , making sure that $N \geq K$
2. Calculate what degree polynomial would be needed ($K - 1$)
3. Calculate modulus (a prime number $> S, K, N$)
4. Populate the coefficient with random numbers less than modulus
5. Create N shares using the coefficients
6. Output the shares and the calculated modulus

Algorithm 1 Encryption

Input: A value S to encrypt, a value K as a threshold, and a value N as the number of shares wanted**Output:** Calculated shares and modulus

```

1: function GENERATEY(polynomial,  $x$ , modulus)
2:    $y := \text{polynomial}[0]$ 
3:   for  $i$  to  $(\text{len}(\text{polynomial})-2)$  do
4:      $y := y + x^{i+1}$ 
5:   end for
6:   return  $y \bmod \text{modulus}$ 
7: end function
8:
9: if  $K > N$  then
10:   End
11: end if
12:
13:  $\text{degree} := K - 1$ 
14:
15: if  $\text{secret} \geq K$  and  $\text{secret} \geq N$  then
16:    $\text{tempMod} := \text{secret}$ 
17: else if  $K \geq \text{secret}$  and  $K \geq N$  then
18:    $\text{tempMod} := K$ 
19: else
20:    $\text{tempMod} := N$ 
21: end if
22:  $\text{modulus} = \text{GREATERPRIME}(\text{tempMod})$ 
23:
24:  $\text{polynomial} := [S]$ 
25: for  $i := 0$  to  $\text{degree}-1$  do
26:    $\text{polynomial.append}(\text{randint}(1, \text{modulus} - 1))$ 
27: end for
28:
29:  $\text{shares} = []$ 
30: for  $i := 0$  to  $N - 1$  do
31:    $\text{point} := []$ 
32:    $x := \text{randint}(1, \mathbb{Z})$ 
33:    $y := \text{GENERATEY}(\text{polynomial}, x, \text{modulus})$ 
34:    $\text{point.append}(x)$ 
35:    $\text{point.append}(y)$ 
36:    $\text{shares.append}(\text{point})$ 
37: end for
38: return ( $\text{shares}$ ,  $\text{modulus}$ )

```

You see a function called GREATERPRIME was used in this Pseudocode, and it will be defined later. In lines 2-11, we have a function that generates a y value given the polynomial coefficients, the x value, and the modulus. By multiplying each coefficient by x to its respective power and adding the y constant, or secret, we are able to generate x 's y point, and that would be one share, of course after taking the modulus of the y to make sure that it is in the field of y values that our polynomial is in.

Algorithm 2 Prime number functions

```

1: function PRIME( $n$ )
2:   for  $i := 0$  to  $\sqrt{|n|} + 1$  do
3:     if  $n \bmod i = 0$  then
4:       return false
5:     end if
6:   return true
7: end for
8: end function
9:
10: function GREATERPRIME(number)
11:   primeNumber := number + 1
12:   while not PRIME(primeNumber) do
13:     primeNumber := primeNumber + 1
14:   end while
15:   return primeNumber
16: end function

```

Here we have how we can calculate our prime number for modulus, where we brute force finding the prime number by continuously adding 1 to the given number that number is prime. To the best of my knowledge, this is the best of finding a prime number.

Lets move on to the fun part, **the decryption**. Note that the modulus must be known to recreate the secret, as to my knowledge, there is no way to calculate the modulus with just the shares. When it comes to mod, remember that we cannot do regular division when under mod and that we would need to find the mod inverse of the divisor then multiply it by the dividend. I am not sure if it is correct, but it works for me, but after getting the Lagrange polynomial's values at $x = 0$, multiply by there respective y values, then added them all up, we have to do the mod of that final value. **Now lets go over the steps for decryption:**

1. Input some number of shares and the modulus
2. Find the value of the Lagrange polynomial for every point at $x = 0$
 - To find the Lagrange polynomial, the mod inverse must be found for the divisor
3. Multiply each value by their respective y value
4. Add up all of the values, then do mod modulus

Algorithm 3 Decryption

Input: share, or a list of points, and modulus
Output: secret, or the decrypted secret from the points

```

1: function MODINVERSE(number, modulus)
2:   inverse := 1
3:   while (number · inverse) mod modulus  $\neq$  1 do
4:     inverse := inverse + 1
5:   end while
6:   return inverse
7: end function
8:
9: function LAGRANGE(position, shares, modulus)
10:  dividend := 1
11:  divisor := 1
12:  for  $i := 0$  to len(shares)−1 do
13:    if  $i = \text{position}$  then
14:      Continue
15:    end if
16:     $x_j := \text{shares}[i][0]$ 
17:     $x_i := \text{shares}[\text{position}][0]$ 
18:    dividend := dividend · ( $-x_j$ )
19:    divisor := divisor · ( $x_i + (-x_j)$ )
20:  end for
21:  inverse := MODINVERSE(divisor, modulus)
22:  lagrangeValue := dividend · inverse
23:  return lagrangeValue
24: end function
25:
26: secret := 0
27: lagrangeValues := []
28: for  $i := 0$  to len(shares)−1 do
29:   IVal := LAGRANGE( $i$ , shares, modulus)
30:   lagrangeValues.append(IVal)
31: end for
32: for  $i := 0$  to len(shares)−1 do
33:   secret := secret + shares[ $i$ ][1] · lagrangeValues[ $i$ ]
34: end for
35: secret := secret mod modulus
36: return secret

```

The first function MODINVERSE() brute forces looking for the mod divisor, where it will loop through numbers until it finds it. The proper way of finding the mod inverse is using the **Extended Euclidean Algorithm**, but chose to just brute force it instead.

Note that I first made the coding implementation of both the encryption and decryption algorithms of SSS, and translated it to Pseudocode. The coding implementation can be found here:

<https://www.jdoodle.com/ga/eAkyDME7ZOQgF91souI%2B7g%3D%3D>

4 Security Analysis

In SSS, it would be impossible to reconstruct the secret without having enough shares to meet the threshold. The modular arithmetic makes it harder to gain some information about the secret.

The assumptions here are that the number of shares N , the threshold $K - 1$, and the modulus is known. No matter what, you cannot reconstruct the secret without having enough shares to meet the threshold. Because of this, the SSS encryption algorithm does not rely on how hard it is to compute the secret, but on a mathematical property.

The point of failure when it comes to SSS, would not be the algorithm itself but on how secure the storage of the shares is, and things like that.

5 Real-world relevance

In the real world, the idea of splitting up a secret among multiple people, and only reconstructing it given enough people to agree, is a concept that is used in digital wallets, vaults, or any other systems where you would want multiple people with permission to access something. One can say that something like nuclear launch codes can also be an example.